

Exercise #3

30. January 2023

Problem 1. (Rounding errors.)

It is known that, for smooth enough functions, the central finite difference approximation of second-order derivatives converges quadratically, that is,

$$\left| \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} - u''(x) \right| = O(h^2).$$

Computationally, however, this is only attainable for a certain range of h . If we keep reducing h , eventually the convergence will break down, since *machine accuracy* (or *machine precision* depending on the source) is usually limited to $O(10^{-16})$ values. Based on that, estimate the mesh size h (in order of magnitude) for which roundoff and approximation errors become comparable.

Problem 2. (Rounding errors)

In this problem we are going to see how round off errors affect the solution to systems of linear equations.

Consider the problem of interpolating the polynomial $f(y) = 1 + y^2$ at the points $y = 0, 1, 2$, using a second degree polynomial. This can be done by solving the system of equations,

$$Ax = b, \quad A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix},$$

where $x = (a_1, a_2, a_3)$ is the coefficients of the resulting polynomial (that is, our interpolating polynomial will look like $a_1 + ya_2 + y^2a_3$), and the vector b has entries $(f(0), f(1), f(2))$. Clearly, the solution to this system of equations is $(1, 0, 1)$.

When computing the solution x on a computer, the right hand side will not be exactly equal to b , but contain some error, $e \in \mathbb{R}^3$ (so in practice we are solving $Ax = b + e$).

The *condition number*, $\kappa(A)$, of the matrix A reflects how errors in the right hand side b propagate into the resulting solution x . It is defined as the maximum ratio of relative error in the right hand side and the in the solution,

$$\kappa(A) = \max_{b, e \neq 0} \frac{|A^{-1}e|}{|A^{-1}b|} \cdot \frac{|b|}{|e|},$$

where $|\cdot|$ denotes the Euclidean norm. It turns out that $\kappa(A)^2$ is equal to the ratio of the largest and smallest eigenvalue of AA^T , so that $\kappa(A)^2 = \lambda_{\max}(AA^T)/\lambda_{\min}(AA^T)$.

- a) (J) Solve the system $Ax = b$ on your computer and compare the solution to $(1, 0, 1)$, using the relative error,

$$\frac{|A^{-1}e|}{|(1, 0, 1)|},$$

where $|\cdot|$ is the Euclidean norm.

Hint. The python function `np.linalg.norm(x, 2)` returns the Euclidean norm of a numpy array x .

- b) Compute the condition number of the matrix A (feel free to use any software to do so). Given that the relative error, $|e|/|b|$, in the right hand side is of order 10^{-10} , estimate the order of the relative error in the solution x .
- c) (J) Change the interpolation points to $0, 0.01, 2$, and make a new system of linear equations for the interpolating polynomial's coefficients. Solve this system on your computer and compare this with $(1, 0, 1)$. Finally compute the condition number of this system's matrix. Still using a relative error in b of order 10^{-10} , what is an upper bound for the order of the relative error in the solution now?
- d) Experiment numerically with different interpolation points in the interval $[0, 2]$, and see if you are able to get an error, $|(1, 0, 1) - A^{-1}b|$ of order 10^{-5} or bigger.

Problem 3. (Fixed-point method)

In this exercise, we will use fixed-point iterations to find the real root of

$$f(x) = 2x^3 - x^2 + 2x - 1 = 0. \quad (1)$$

To apply the fixed-point algorithm, we first need to rewrite Eq. (1) as $x = g(x)$, but this representation is not unique. For example, we can write

$$x = \frac{-2x^3 + x^2 + 1}{2} := g_1(x), \quad \text{or} \quad x = \sqrt[3]{\frac{x^2 - 2x + 1}{2}} := g_2(x).$$

In fact, the algorithm's convergence will depend on whether we choose g_1 or g_2 as our $g(x)$. We will experiment with both versions, and you can use the Jupyter notebook *05-Nonlinear-eqs.ipynb* for help.

- Find the expressions for $g'_1(x)$ and $g'_2(x)$ and, based on that, answer: would you rather select g_1 or g_2 as your $g(x)$? Why?
- (J) Set $x^{(0)} = 0.9$ and $g(x) = g_1(x)$, and iterate until $|x^{(k+1)} - x^{(k)}| < 10^{-6}$. What is the solution obtained, and how many iterations are needed to reach that?
- (J) Now, do the same for $g(x) = g_2(x)$. How many iterations are now needed?

Problem 4. (Fixed-point iterations)

In pipeline design for oil transport, pressure losses must be carefully estimated. They are directly proportional to a positive friction factor f_D , whose inverse square root $x := 1/\sqrt{f_D}$ is given by a non-linear equation. In fact, it is known that x depends only on the Reynolds number R (look up “*Reynolds number*” online, if you are interested in fluid mechanics). For a turbulent flow at a given R , the equation to find x is

$$x + 1.93 \ln(1.32x/R) = 0,$$

in which $\ln(x)$ denotes the natural logarithm. In this exercise, we will set $R = 5000$ and use fixed-point iterations to approximate x , according to

$$x^{(k+1)} = g(x^{(k)}), \quad \text{with } g(x) := -1.93 \ln(1.32x/5000). \quad (2)$$

Let us also define the iteration error as $e_k := |x^{(k)} - x^{(k-1)}|$. Based on that, you are asked to:

- a) Compute $g'(x)$ and use the result to determine whether $g(x)$, $x > 0$, is an increasing, decreasing or a non-monotonic function.
- b) Calculate the maximum and minimum values of $g(x)$ in the interval $x \in [e, e^3]$.
- c) Determine the maximum value L of $|g'(x)|$ in the interval $[e, e^3]$.
- d) For $x^{(0)} = e^2$, perform the first fixed-point iteration for the solution of Eq. (2).
- e) Based on the values of L , $x^{(0)}$ and $x^{(1)}$, find an upper bound for the number of iterations required to reach a tolerance of 10^{-3} .